

# 命题逻辑的推理 II

---

祝恩  
计算机学院  
国防科技大学



# 学习内容

---

## 命题逻辑的推理：

A. 几种证明方法

B. 模型检验

C. 逻辑等价

D. 基于推理规则

E. 归结推理

F. Horn子句和限定子句

G. 前向链接和反向链接

## $KB \models \alpha$ iff...

- $KB \models \alpha$
- iff  $M(KB) \subseteq M(\alpha)$
- iff  $(KB \wedge \neg \alpha)$  is unsatisfiable
- iff  $KB \Rightarrow \alpha$  is valid
- iff  $KB, \neg \alpha \models \text{False}$

- to prove  $KB \models \alpha$  :
  - From  $KB$ , derive  $\alpha$
  - From  $KB, \neg \alpha$ , derive  $\text{False}$

inference rules  
resolution



## E. 归结推理

## 单元归结推理规则

□ Resolution inference rule (for CNF)

$$\frac{l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, m}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k}$$

where  $l_i$  and  $m$  are complementary literals.

A literal is either an atomic sentence or the negated atomic sentence, e.g.:  $P, \neg P$

$$\frac{\neg A \vee B, A}{B}$$

$$\frac{\neg A \vee B \vee C, A}{B \vee C}$$

## 归结

- $KB \models \alpha$ , iff  $KB \Rightarrow \alpha$  is valid
- $(A \vee \neg B) \wedge (B \vee C) \Rightarrow A \vee C$  is valid
- $(A \vee \neg B) \wedge (B \vee C) \models A \vee C$
- the following inference rule is sound:

$$\frac{A \vee \neg B, B \vee C}{A \vee C}$$

推理算法i从KB中导出的语句 $\alpha$ 都是KB蕴涵的语句，该推理算法被称为可靠的(sound)或真值保持的(true-preserving)

$$\frac{\neg A \Rightarrow \neg B, \neg B \Rightarrow C}{\neg A \Rightarrow C}$$

## 互补文字

- A *literal* (文字) is either an atomic sentence or the negated atomic sentence, e.g.:

$$P, \neg P$$

- Two literals are **complementary** (互补) if one is the negation of the other, e.g.:

$$P \text{ and } \neg P$$

## 子句和CNF

### □ Clause (子句)

- disjunction of literals
- E.g.,  $A \vee \neg B$
- $A, \neg A$ : unit clause
- NIL: empty clause (空子句)

### □ CNF: Conjunctive Normal Form

- conjunction of clauses
- E.g.,  $(A \vee \neg B) \wedge (B \vee C)$

$\frac{A \vee \neg B, B \vee C}{A \vee C}$  : Basic intuition, resolve  $B, \neg B$  to  $A \vee C$



## 转换为CNF

- $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
- $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
- $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

1. Eliminate  $\Leftrightarrow$ , replacing  $\alpha \Leftrightarrow \beta$  with  $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$ .
2. Eliminate  $\Rightarrow$ , replacing  $\alpha \Rightarrow \beta$  with  $\neg \alpha \vee \beta$ .
3. Move  $\neg$  inwards using de Morgan's rules and double-negation:
4. Apply distributivity law ( $\wedge$  over  $\vee$ ) and flatten:

# 归结推理规则

## □ Resolution inference rule (for CNF)

$$\frac{l_1 \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_n}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where  $l_i$  and  $m_j$  are complementary literals.

$$\frac{P, \neg P \vee Q}{Q}$$

$$\frac{P \vee Q, \neg P \vee Q}{Q}$$

去除文字的多余  
副本称为归并  
(factoring)

$$\frac{\neg P \vee Q, \neg Q \vee R}{\neg P \vee R}$$

$$\frac{P, \neg P}{\text{NIL}}$$

## 全归结、单元归结

*Resolution rule:*

$$\frac{l_1 \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_n}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where  $l_i$  and  $m_j$  are complementary literals

称为全归结。

$$\frac{l_1 \vee \cdots \vee l_k, \quad m}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k}$$

where  $l_i$  and  $m$  are complementary literals

称为单元归结。

## 如何通过归结进行证明

□ How to prove  $KB \models \alpha$  using resolution?

□ consider:

□  $KB \models \alpha$

iff  $(KB \wedge \neg \alpha)$  is unsatisfiable

iff  $KB, \neg \alpha \models \text{false}$

$$\frac{P, \neg P}{\text{NIL}}$$

## 如何通过归结进行证明

1. 定义命题符号，将问题表示成逻辑语句：写出问题的前提条件语句（KB）和要证明的结论（查询语句a）
2. 将 $\{KB, \neg a\}$ 转换成子句集合。
3. 反复进行归结，直到归结出NIL。



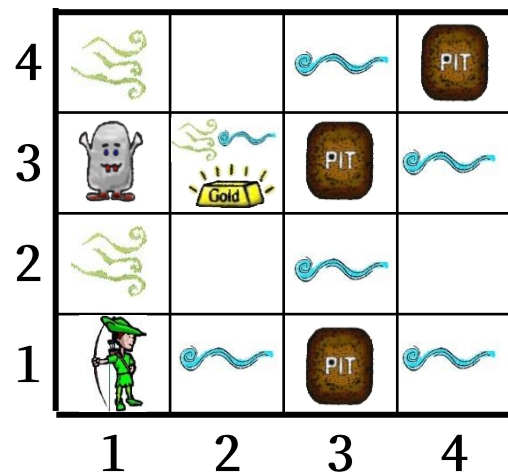
# wumpus问题的归结推理

- Pits cause breezes in adjacent squares
  - $R_1: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
  - $R_2: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- Start
  - $R_3: \neg P_{1,1}$
- Perceiving
  - $R_4: \neg B_{1,1}$
  - $R_5: B_{2,1}$

□  $KB \models \neg P_{1,2} ?$

□ CNF representation of KB ...

□ resolution ...



- $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
- $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
- $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$





# wumpus问题的归结推理

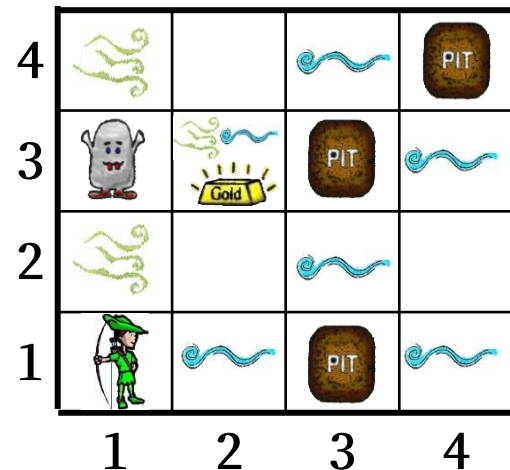
- Pits cause breezes in adjacent squares
  - $R_1: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
  - $R_2: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

- Start
  - $R_3: \neg P_{1,1}$

- Perceiving
  - $R_4: \neg B_{1,1}$
  - $R_5: B_{2,1}$

$KB \models \neg P_{1,2} ?$

- CNF from R1:
  - $R6: \neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$
  - $R7: B_{1,1} \vee \neg P_{1,2}$
  - $R8: B_{1,1} \vee \neg P_{2,1}$
- CNF from R2:
  - $R9: \neg B_{2,1} \vee P_{1,1} \vee P_{2,2} \vee P_{3,1}$
  - $R10: B_{2,1} \vee \neg P_{1,1}$
  - $R11: B_{2,1} \vee \neg P_{2,2}$
  - $R12: B_{2,1} \vee \neg P_{3,1}$



- Negation of conclusion
  - $R13: P_{1,2}$
- Inference:
  - $R14: B_{1,1} \quad 7,13$
  - $R15: \text{NIL} \quad 4,14$



## 归结推理算法

□ Proof by contradiction, showing  $KB \wedge \neg \alpha$  is unsatisfiable

**function** PL-RESOLUTION( $KB, \alpha$ ) **returns** true or false

**inputs:**  $KB$ , the knowledge base, a sentence in propositional logic  
 $\alpha$ , the query, a sentence in propositional logic

**clauses**  $\leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$

**new**  $\leftarrow \{\}$

**loop do**

**for each pair of clauses**  $C_i, C_j$  **in** **clauses** **do**

resolvents  $\leftarrow$  PL-RESOLVE( $C_i, C_j$ )

**if** resolvents contains the empty clause **then return** true

**new**  $\leftarrow$  **new**  $\cup$  resolvents

**if** **new**  $\subseteq$  clauses **then return** false

clauses  $\leftarrow$  clauses  $\cup$  new



## 实例

证明 $\{P \vee Q, \neg P \vee Q, P \vee \neg Q, \neg P \vee \neg Q\}$ 是不可满足的

|                         |     |          |          |                          |      |      |
|-------------------------|-----|----------|----------|--------------------------|------|------|
| 1) $P \vee Q$           |     | clauses1 | clauses2 | 13) $P \vee Q$           |      | new2 |
| 2) $\neg P \vee Q$      |     |          |          | 14) $P \vee Q$           |      |      |
| 3) $P \vee \neg Q$      |     |          |          | 15) $P \vee Q$           |      |      |
| 4) $\neg P \vee \neg Q$ |     |          |          | 16) $P \vee Q$           |      |      |
| 5) $Q$                  | 1,2 | new1     |          | 17) $Q$                  | 1,11 |      |
| 6) $P$                  | 1,3 |          |          | 18) $P$                  | 1,12 |      |
| 7) $P \vee \neg P$      | 1,4 |          |          | 19) $Q$                  | 2,6  |      |
| 8) $Q \vee \neg Q$      | 1,4 |          |          | 20) $\neg P \vee Q$      | 2,7  |      |
| 9) $Q \vee \neg Q$      | 2,3 |          |          | 21) $\neg P \vee Q$      | 2,8  |      |
| 10) $P \vee \neg P$     | 2,3 |          |          | 22) $\neg P \vee Q$      | 2,9  |      |
| 11) $\neg P$            | 2,4 |          |          | 23) $\neg P \vee Q$      | 2,10 |      |
| 12) $\neg Q$            | 3,4 |          |          | 24) $\neg P$             | 2,12 |      |
|                         |     |          |          | 25) $P$                  | 3,5  |      |
|                         |     |          |          | 26) $P \vee \neg Q$      | 3,7  |      |
|                         |     |          |          | 27) $P \vee \neg Q$      | 3,8  |      |
|                         |     |          |          | 28) $P \vee \neg Q$      | 3,9  |      |
|                         |     |          |          | 29) $P \vee \neg Q$      | 3,10 |      |
|                         |     |          |          | 30) $\neg Q$             | 3,11 |      |
|                         |     |          |          | 31) $\neg P$             | 4,5  |      |
|                         |     |          |          | 32) $\neg Q$             | 4,6  |      |
|                         |     |          |          | 33) $\neg P \vee \neg Q$ | 4,7  |      |
|                         |     |          |          | 34) $\neg P \vee \neg Q$ | 4,8  |      |
|                         |     |          |          | 35) $\neg P \vee \neg Q$ | 4,9  |      |
|                         |     |          |          | 36) $\neg P \vee \neg Q$ | 4,10 |      |
|                         |     |          |          | 37) $Q$                  | 5,8  |      |
|                         |     |          |          | 38) $Q$                  | 5,9  |      |
|                         |     |          |          | 39) NIL                  | 5,12 |      |

## 命题逻辑归结推理的可靠性

- 因为归结推理规则是可靠的，所以归结推理算法是可靠的。

```

function PL-RESOLUTION(KB, $\alpha$ ) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
            $\alpha$ , the query, a sentence in propositional logic
  clauses  $\leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$ 
  new  $\leftarrow \{\}$ 
  loop do
    for each pair of clauses  $C_i, C_j$  in clauses do
      resolvents  $\leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if resolvents contains the empty clause then return true
      new  $\leftarrow$  new  $\cup$  resolvents
    if new  $\subseteq$  clauses then return false
  clauses  $\leftarrow$  clauses  $\cup$  new
  
```

归结推理规则与完备的搜索算法结合得到的算法是完备的。  
如果  $KB \models \alpha$ ，**PL-RESOLUTION**能够停止并返回**true**。

是可靠的，也是完备的，但低效



## F. Horn子句与限定子句

## Horn子句

- 至多有一个正文字的析取式称为Horn子句。  
(没有正文字 或者 恰好一个正文字)
- Horn子句在归结下是封闭的：对两个Horn子句进行归结，归结结果依然是Horn子句。

## 限定子句

- 至多有一个正文字的析取式称为Horn子句。  
(没有正文字 或者 恰好一个正文字)
- 其中:
  - 恰好有一个正文字的析取式称为限定子句。
    - 例如 $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ 是限定子句,
    - 单个正文字的子句称为事实。
  - 没有正文字的子句称为目标子句。

## 限定子句

只包含限定子句的知识库对推理有三个好处：

- 每个限定子句可写为**蕴含式**，蕴含式更易理解
  - $\neg L_{1,1} \vee \neg Breeze \vee B_{1,1}$
  - $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$
- 使用限定子句的**推理**可使用前向连接和后向连接
- 用限定子句判定蕴涵需要的**时间**与知识库大小成**线性关系**



## G. 前向链接与后向链接

## 前向链接

KB

$P \Rightarrow Q$   
 $L \wedge M \Rightarrow P$   
 $B \wedge L \Rightarrow M$   
 $A \wedge P \Rightarrow L$   
 $A \wedge B \Rightarrow L$   
A  
B

$\models ?$

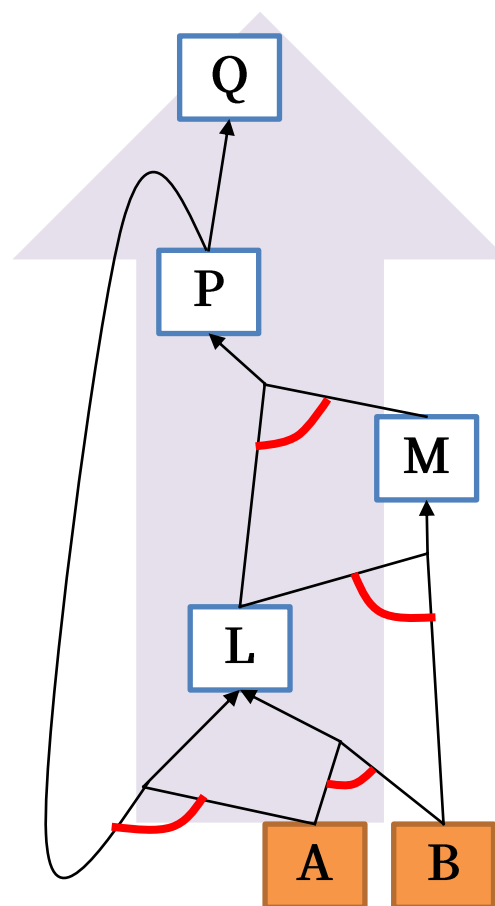
判断KB蕴涵哪些结论



## 前向链接

KB

$P \Rightarrow Q$   
 $L \wedge M \Rightarrow P$   
 $B \wedge L \Rightarrow M$   
 $A \wedge P \Rightarrow L$   
 $A \wedge B \Rightarrow L$   
A  
B



## 前向链接

Loop:

- 从Agenda弹出一个符号;
- 若它的inferred值为T, 则进入下一循环;
- 将该符号的inferred值变为T;
- 将前提含有该符号的语句的count值减1, count值变为0的语句的结论加入Agenda。

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

|                                     | start | B | A | L | M | P  | L | Q |   |
|-------------------------------------|-------|---|---|---|---|----|---|---|---|
| Agenda                              | A,B   | A | L | M | P | QL | Q |   |   |
| Inferred[A]                         | F     |   | T |   |   |    |   |   | T |
| Inferred[B]                         | F     | T |   |   |   |    |   |   | T |
| Inferred[L]                         | F     |   |   | T |   |    |   |   | T |
| Inferred[M]                         | F     |   |   |   | T |    |   |   | T |
| Inferred[P]                         | F     |   |   |   |   | T  |   |   | T |
| Inferred[Q]                         | F     |   |   |   |   |    |   | T | T |
| Count[P $\Rightarrow$ Q]            | 1     |   |   |   |   | 0  |   |   | 0 |
| Count[L $\wedge$ M $\Rightarrow$ P] | 2     |   |   | 1 | 0 |    |   |   | 0 |
| Count[B $\wedge$ L $\Rightarrow$ M] | 2     | 1 |   | 0 |   |    |   |   | 0 |
| Count[A $\wedge$ P $\Rightarrow$ L] | 2     |   | 1 |   |   | 0  |   |   | 0 |
| Count[A $\wedge$ B $\Rightarrow$ L] | 2     | 1 | 0 |   |   |    |   |   | 0 |
| Count[A]                            | 0     |   |   |   |   |    |   |   | 0 |
| Count[B]                            | 0     |   |   |   |   |    |   |   | 0 |

# 前向链接

| p                |     | B | A | L | M |  |  |  |  |
|------------------|-----|---|---|---|---|--|--|--|--|
| Agenda           | A,B | A | L | M |   |  |  |  |  |
| Inferred[A]      | F   |   | T |   |   |  |  |  |  |
| Inferred[B]      | F   | T |   |   |   |  |  |  |  |
| Inferred[L]      | F   |   |   | T |   |  |  |  |  |
| Inferred[M]      | F   |   |   |   |   |  |  |  |  |
| Inferred[P]      | F   |   |   |   |   |  |  |  |  |
| Inferred[Q]      | F   |   |   |   |   |  |  |  |  |
| Count[P⇒Q]       | 1   |   |   |   |   |  |  |  |  |
| Count[L ∧ M ⇒ P] | 2   |   |   | 1 |   |  |  |  |  |
| Count[B ∧ L ⇒ M] | 2   | 1 |   | 0 |   |  |  |  |  |
| Count[A ∧ P ⇒ L] | 2   |   | 1 |   |   |  |  |  |  |
| Count[A ∧ B ⇒ L] | 2   | 1 | 0 |   |   |  |  |  |  |
| Count[A]         | 0   |   |   |   |   |  |  |  |  |
| Count[B]         | 0   |   |   |   |   |  |  |  |  |

## 判断KB是否蕴涵q的前向链接算法

Function PL-FC-Entails?(KB,q) returns *true* or *false*

Inputs: KB, the knowledge base, a set of propositional definite clauses  
q, the query, a proposition symbol

Count ← a table, where Count[c] is the number of symbols in c’s premise

Inferred ← a table, where Inferred[s] is initially false for all symbols

Agenda ← a queue of symbols, initially symbols known to be true in KB

While Agenda is no empty do

    p ← POP(Agenda)

    if p=q then return *true*

    if Inferred[p]=*false* then

        Inferred[p] ← *true*

        for each clause c in KB where p is in c.Premise do

            decrement Count[c]

            if Count[c]=0 then add c.Conclusion to Agenda

Return *false*

- Agenda记录已知为真但未被“处理”的符号。
- inferred记录已经推出的符号集合。
- count记录每个蕴含式还有多少前提未知。当Agenda中的一个符号p被处理，对前提中出现p的每个蕴含式，它的count减去1，如果值变为0，将它的结论添加到Agenda中。

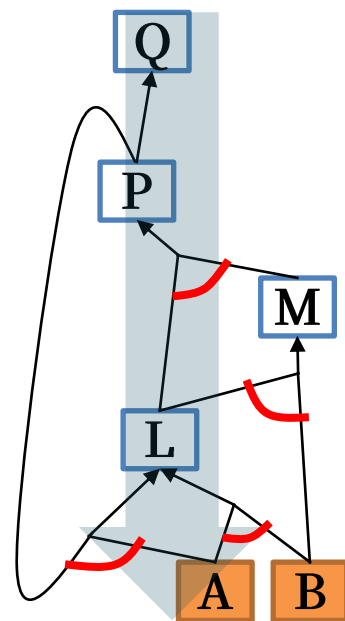
## 前向链接：数据驱动

- *Agenda*记录已知为真但未被“处理”的符号。
- *inferred*记录已经推出的符号集合。
- *count*记录每个蕴含式还有多少前提未知。当*Agenda*中的一个符号 $p$ 被处理，对于前提中出现 $p$ 的每个蕴含式，它的*count*减去1，如果值变为0，将它的结论添加到*Agenda*中。

□ 前向链接是数据驱动推理的实例——推理是从已知数据开始的。是可靠的，完备的。

## 后向链接：目标制导

- 后向链接算法从查询 $q$ 开始进行推理。如果查询 $q$ 已知为真，那么无需进行任何操作。否则，寻找知识库中那些以 $q$ 为结论的蕴含式。如果其中某个蕴含式的所有前提都能证明为真（递归），则 $q$ 为真。与前向链接一样，有效实现的时间复杂度是线性的。
- 后向链接是一种目标制导的推理形式。
- 后向链接算法和与或树搜索相同。



$P \Rightarrow Q$   
 $L \wedge M \Rightarrow P$   
 $B \wedge L \Rightarrow M$   
 $A \wedge P \Rightarrow L$   
 $A \wedge B \Rightarrow L$   
A  
B

查询P

**Thanks**

THANKS